help me do the database structure i have :

{
"id": "string",
"email": "user@example.com",
"first_name": "string",
"last_name": "string",
"role": "string",
"workspace_id": "string",
"created_at": "2026-06-08T15:14:27.532Z",
"profile_image_url": "string",
"access_token": "string",

Show more

Based on your Firebase structure image and your requirement:

- One **Workspace (Office)** can have many employees.
- User logs in with **email + password + workspace_id**.
- After login, the system loads office configuration (geofence, policy, holidays, etc.) from the user's workspace.
- Admin can create employees inside their workspace.

I would redesign your database like this:

users

JSON

```
{
  "_id": "ObjectId",
  "email": "user@example.com",
  "password_hash": "hashed_password",
  "first_name": "Winner",
  "last_name": "Yun",
  "role": "admin",
  "workspace_id": "ws_001",
```

```
  "department": "IT",
  "gender": "male",

  "profile_image_url": "https://...",

  "face_biometrics_id": "face_001",

  "status": "active",

  "created_at": "2026-06-08T15:14:27.532Z",
  "updated_at": "2026-06-08T15:14:27.532Z"
}
```

---

workspaces (offices)

Equivalent to your **offices** collection.

JSON

```
{
  "_id": "ws_001",

  "workspace_name": "ABC Company",
  "office_name": "Main Office",

  "department": [
    "IT",
    "HR",
    "Finance"
  ],

  "admin_id": "user_001",

  "geofence_id": "geo_001",
  "policy_id": "policy_001",

  "status": "active",

  "created_at": "2026-06-08T15:14:27.532Z"
}
```

---

geofences

JSON

```
{
  "_id": "geo_001",

  "workspace_id": "ws_001",

  "group_name": "Main Office",

  "address": "Phnom Penh",

  "center": {
    "lat": 11.5564,
    "lng": 104.9282
  },

  "radius_meters": 100
}
```

---

policies

JSON

```
{
  "_id": "policy_001",

  "workspace_id": "ws_001",

  "check_in_start": "08:00",
  "check_in_end": "09:00",

  "late_buffer_minutes": 10,

  "deadline_scan_minutes": 15,

  "annual_leave_limit": 18,
  "sick_leave_limit": 12
}
```

---

holidays

JSON

```
{
  "_id": "holiday_001",

  "workspace_id": "ws_001",
```

```
  "date": "2026-01-01",
  "description": "New Year"
}
```

---

face_biometrics

JSON

```
{
  "_id": "face_001",

  "user_id": "user_002",

  "face_count": 5,

  "embedding_vector": [
    0.12,
    -0.23,
    0.88
  ],

  "registered_at": "2026-06-08T15:14:27.532Z",

  "status": "active"
}
```

For MongoDB, store the embedding vector as an array of floats.

---

attendance_records

JSON

```
{
  "_id": "att_001",

  "user_id": "user_002",
  "workspace_id": "ws_001",

  "date": "2026-06-08",

  "check_in": "2026-06-08T08:02:10Z",
  "check_out": "2026-06-08T17:00:00Z",

  "status": "present",

  "location": {
```

```
    "lat": 11.5564,
    "lng": 104.9282
  },

  "face_verified": true,

  "mock_location_detected": false,

  "created_at": "2026-06-08T08:02:10Z"
}
```

---

leave_requests

JSON

```
{
  "_id": "leave_001",

  "user_id": "user_002",

  "workspace_id": "ws_001",

  "type": "annual",

  "reason": "Family trip",

  "start_date": "2026-06-10",
  "end_date": "2026-06-12",

  "status": "pending",

  "approved_by": null,

  "created_at": "2026-06-08T15:14:27.532Z"
}
```

Login Response

Your current login response is good:

JSON

```
{
  "id": "user_001",
  "email": "admin@company.com",
  "first_name": "Winner",
```

```
  "last_name": "Yun",
  "role": "admin",
  "workspace_id": "ws_001",
  "profile_image_url": "https://...",
  "access_token": "jwt_token",
  "token_type": "bearer"
}
```

After login:

1. JWT contains:

```JSON
{
  "sub": "user_001",
  "workspace_id": "ws_001",
  "role": "admin"
}
```

2. Frontend calls:

```http
GET /workspace/current
```

3. Backend uses `workspace_id` from JWT and returns:

```JSON
{
  "workspace": {},
  "geofence": {},
  "policy": {},
  "holidays": []
}
```

This is a clean SaaS architecture where:

```
Workspace (Company)
        |
        ├── Users
        ├── Geofence
        ├── Policy
        ├── Holidays
        ├── Attendance Records
        └── Leave Requests
```

This structure scales well if later you have 100+ companies using the same system.

write me in document

You can put this directly into your project documentation (System Design / Database Design section).

Database Structure Design

Overview

The system follows a multi-workspace architecture where each company or organization is represented as a Workspace. Every employee belongs to a specific workspace and can only access data associated with that workspace. Administrators are responsible for managing employees, attendance policies, geofence settings, holidays, and leave requests within their workspace.

When a user logs into the system using their email, password, and workspace ID, the system retrieves all configuration data associated with the corresponding workspace, including office information, geofence settings, attendance policies, and holidays.

---

1. Users Collection

The Users collection stores employee and administrator account information.

Fields

Field	Type	Description
id	String	Unique user identifier
email	String	User email address
password_hash	String	Encrypted password
first_name	String	User first name

last_name	String	User last name
role	String	admin or employee
workspace_id	String	Associated workspace
department	String	User department
profile_image_url	String	Profile picture URL
face_biometrics_id	String	Linked face biometric record
status	String	active or inactive
created_at	DateTime	Account creation date
updated_at	DateTime	Last update timestamp

---

2. Workspaces Collection

The Workspaces collection represents a company or office.

Fields

Field	Type	Description
workspace_id	String	Unique workspace identifier
workspace_name	String	Company name
office_name	String	Office name
admin_id	String	Workspace administrator
geofence_id	String	Linked geofence configuration
policy_id	String	Linked attendance policy
status	String	Workspace status
created_at	DateTime	Creation date

Relationships

One workspace can contain:

- Multiple users
- One geofence configuration
- One attendance policy
- Multiple holidays
- Multiple attendance records
- Multiple leave requests

---

3. Geofences Collection

Stores GPS boundary settings used for attendance validation.

Fields

Field	Type	Description
geofence_id	String	Unique geofence identifier
workspace_id	String	Related workspace
group_name	String	Geofence name
address	String	Office address
center_lat	Double	Latitude coordinate
center_lng	Double	Longitude coordinate
radius_meters	Integer	Allowed radius in meters

---

4. Policies Collection

Stores attendance and leave policies for each workspace.

Fields

Field	Type	Description

policy_id	String	Unique policy identifier
workspace_id	String	Related workspace
check_in_start	Time	Earliest check-in time
check_in_end	Time	Latest check-in time
late_buffer_minutes	Integer	Grace period
deadline_scan_minutes	Integer	Attendance deadline
annual_leave_limit	Integer	Annual leave allowance
sick_leave_limit	Integer	Sick leave allowance

---

5. Holidays Collection

Stores official holidays for a workspace.

Fields

Field	Type	Description
holiday_id	String	Unique holiday identifier
workspace_id	String	Related workspace
holiday_date	Date	Holiday date
description	String	Holiday description

---

6. Face Biometrics Collection

Stores employee face embeddings used for face recognition.

Fields

Field	Type	Description

face_biometrics_id	String	Unique biometric identifier
user_id	String	Related user
face_count	Integer	Number of registered faces
embedding_vector	Array	Face feature vector
registered_at	DateTime	Registration timestamp
status	String	active or inactive

---

7. Attendance Records Collection

Stores daily attendance information.

Fields

Field	Type	Description
attendance_id	String	Unique attendance record
user_id	String	Employee identifier
workspace_id	String	Associated workspace
attendance_date	Date	Attendance date
check_in_time	DateTime	Check-in timestamp
check_out_time	DateTime	Check-out timestamp
status	String	present, late, absent
latitude	Double	Attendance latitude
longitude	Double	Attendance longitude
face_verified	Boolean	Face verification result
mock_location_detected	Boolean	GPS spoofing detection

created_at	DateTime	Record creation time

---

8. Leave Requests Collection

Stores leave applications submitted by employees.

Fields

Field	Type	Description
leave_request_id	String	Unique leave identifier
user_id	String	Employee identifier
workspace_id	String	Related workspace
leave_type	String	Annual or Sick Leave
reason	String	Leave reason
start_date	Date	Leave start date
end_date	Date	Leave end date
status	String	pending, approved, rejected
approved_by	String	Administrator identifier
created_at	DateTime	Submission timestamp

Authentication Flow

1. User enters email, password, and workspace ID.
2. The system validates the credentials.
3. A JWT access token is generated.
4. The token contains:
    - User ID
    - Workspace ID
    - User Role

5. After login, the system loads:
    - Workspace information
    - Geofence configuration
    - Attendance policy
    - Holidays
    - Employee permissions

This design ensures complete data isolation between different organizations while supporting scalable multi-tenant deployment.

This format is suitable for **Chapter 3: System Design → Database Design** in a thesis or project report.